

Introducción:

La librería de QT es una potente librería de programación de interfaces graficas provista por la empresa americana "The QT Company". Es una librería con licencias de código abierto y licencias con fines comerciales disponibles. Es una de las principales maneras de desarrollar apps complejas en el mundo de código abierto, y se eligió por la interfaz de compatibilidad con Python que ofrece la librería PyQt.

1. Instalación del DesignerQT

El DesignerQT se puede descargar en conjunto con la librería con de QT <https://www.qt.io/download>

O con el link directo <https://build-system.fman.io/qt-designer-download>

Qt Designer Download

Install Qt Designer on Windows or Mac.
Tiny download: Only 40MB!

Many people want to use Qt Designer without having to download gigabytes of other software. Here are small, standalone installers of Qt Designer for Windows and Mac:



Windows (31 MB)



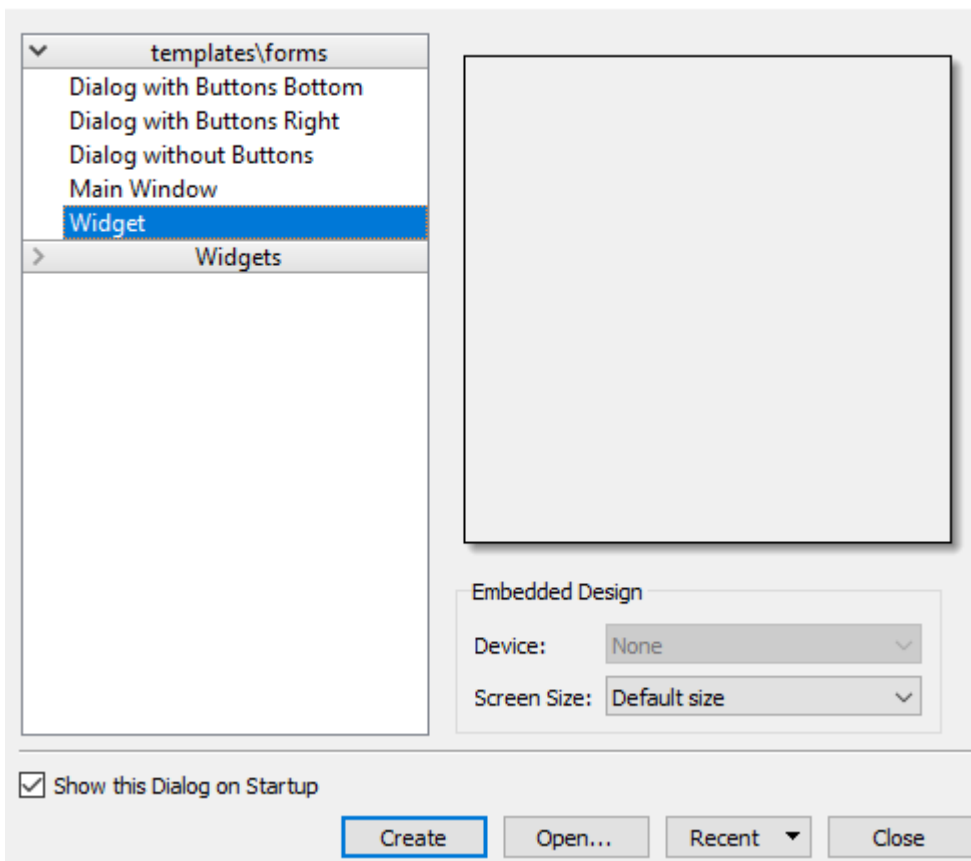
Mac (40 MB)

If you encounter any problems, please just [send us an email](#). We'd be happy to help.

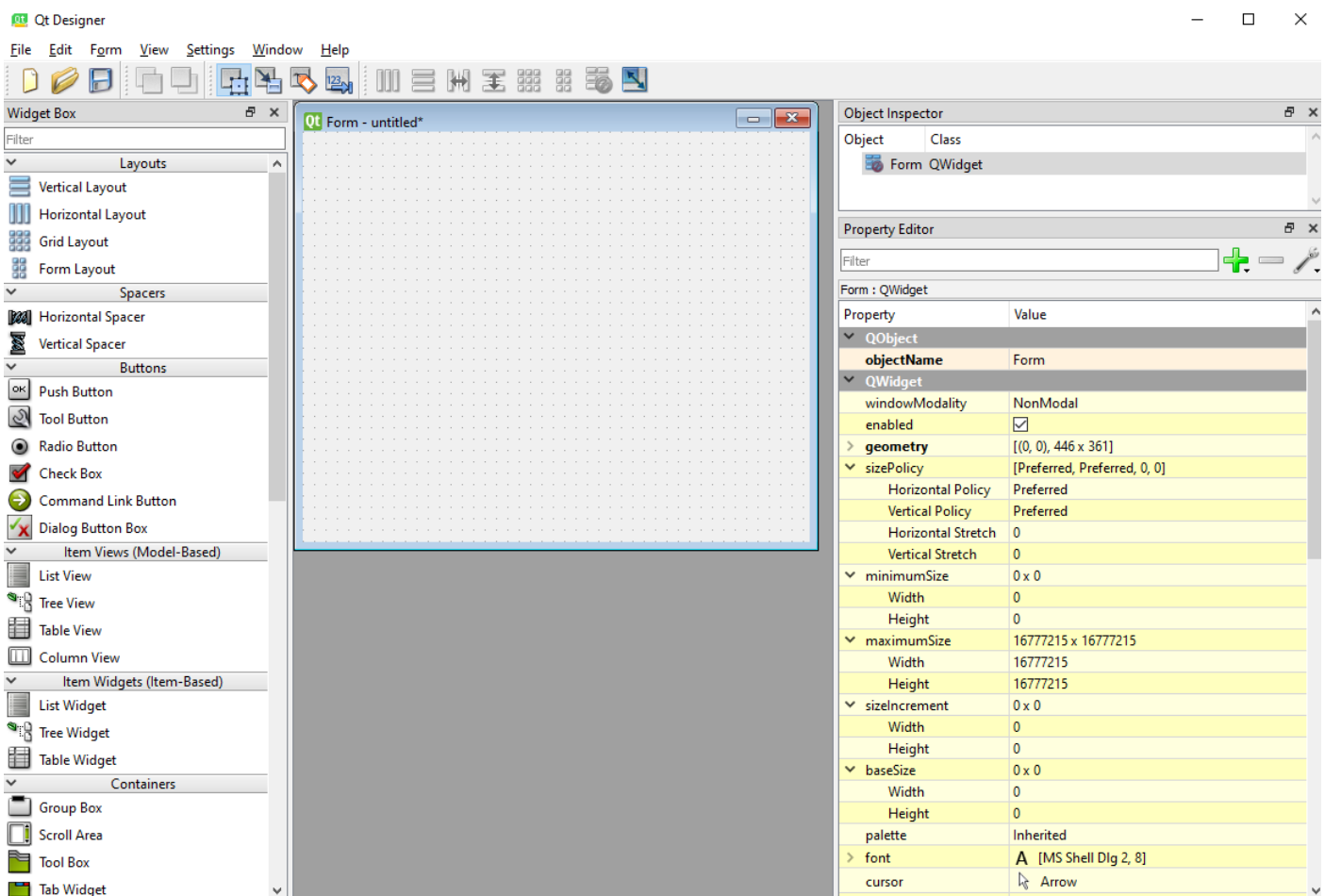
El Instalador es simple y se puede instalar el programa simplemente dándole next al programa de instalación que da el link.

2. Crear un Widget Nuevo

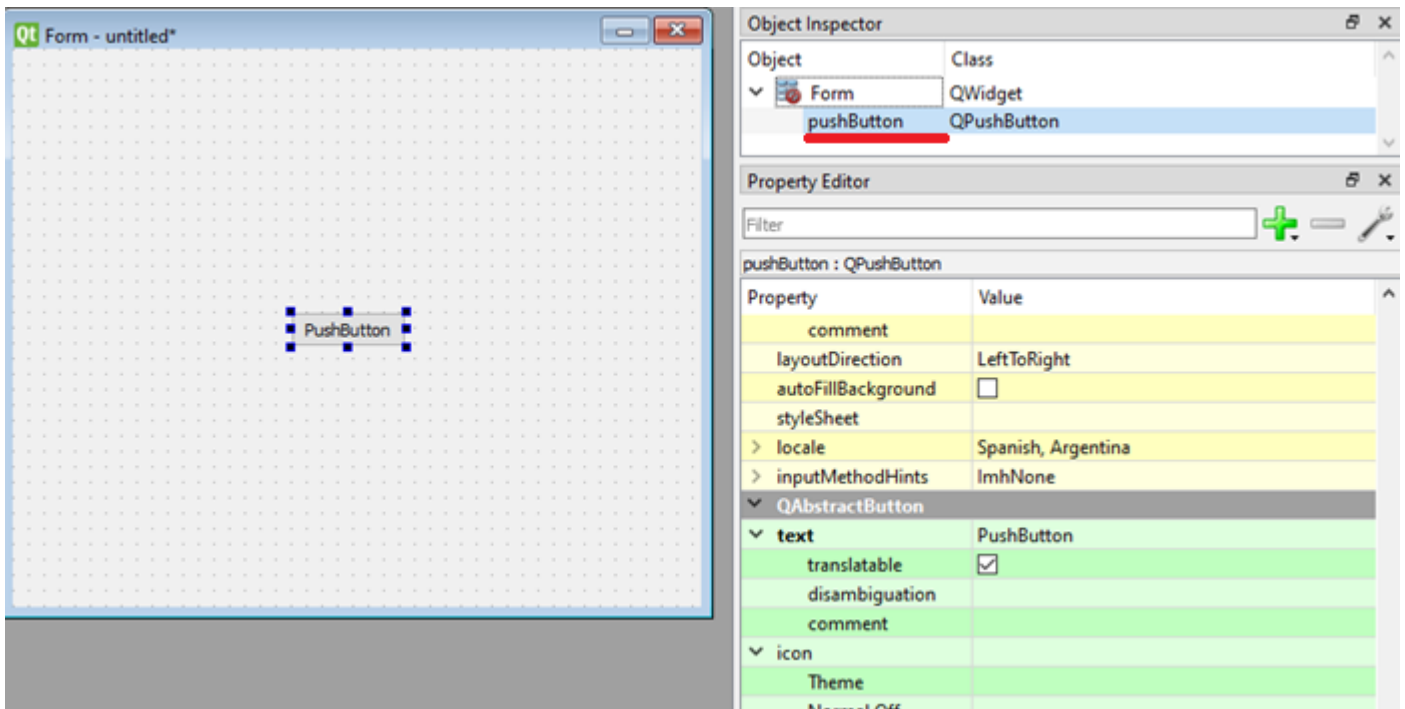
Cuando se abre el programa te da las opciones de crear un archivo .ui nuevo



La ventana que surge al apretar “Create” es la siguiente:

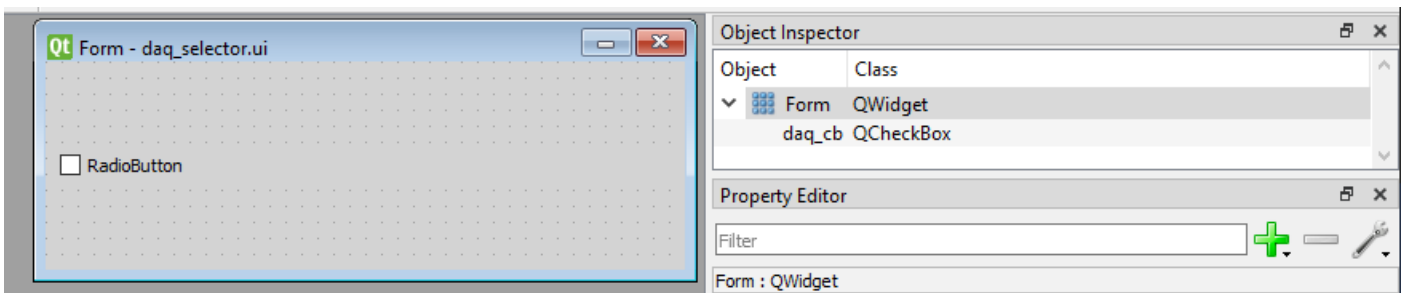


La interfaz para agregar elementos es un simple sistema de drag and drop:



Aca se muestra un ejemplo de un botón. En el código, el nombre que va a cargar lantz es el que aparece subrayado, y haciendo doble click se puede remplazar.

Ejemplo del seleccionador del DAQ:



Estos programa guarda las interfaces a un archivo .ui que tiene que ser cargado por Python para interactuar.

3. Cargar el Widget desde Python con Lantz

Al inicializarse la clase Frontend de Lantz va a buscar el archivo indicado por la tupla `gui = ("Carpeta", "archivo")` y lo utiliza para formar el GUI (Grafical User Interface, o Interfaz Gráfica del Usuario). Este es un ejemplo del daq selector cargando el widget anterior.

```
class DaqSelector(Frontend):
    gui = ("DAQ", "daq_selector.ui")
```

Para mostrar el widget en necesario cargarlo en una aplicación. Hay dos métodos para esto:

1) Inicializar una aplicación nueva. Esto se puede hacer a través del método `start_gui_app` de lantz como se usa en `main.py`. Este método toma como parámetro una clase Frontend, y la utiliza como base para la aplicación, cargando ese gui como interfaz de la aplicación. Ejemplo:

```
start_gui_app(app, ProgramWindow, qapp)
```

2) Cargarlo en una aplicación que ya esta corriendo. Esto se hace a travez del método `addWidget()`, que permite expandir una interfaz de QT con nuevos elementos.

Ejemplo:

```
class ProgramWindow(Frontend):
    gui = ("frontend", "UI", "main.ui")

    def setupUi(self):
        self.selector_frontend = SelectorWindow()
        self.widget.main_lt.addWidget(self.selector_frontend)
```

En este código, se carga el GUI “main.ui” y cuando se inicializa, se le agrega el SelectorWindow como un widget en su interior.

4. Interactuar con QT desde Python

La mejor manera de interactuar en el código con el gui es a través del método setupUi. Este método permite prepararlo para presentarlo por primera vez. Cada widget presente en el archivo se guarda en self.widget.”nombre del widget”. En este ejemplo se carga y presenta el widget del ejemplo anterior:

```
class DaqSelector(Frontend):
    gui = ("DAQ", "daq_selector.ui")

    def setupUi(self):
        self.widget.daq_cb.setText(locale.get("virtual_daq", "str_virtual_daq"))
        if config_file["PREVIOUS INSTRUMENTS"]["daq"] == "Virtual":
            self.widget.daq_cb.setCheckState(Qt.Checked)
        else:
            self.widget.daq_cb.setCheckState(Qt.Unchecked)
```

Línea por línea, este código

- 1) Declara la clase DaqSelector, que hereda de la clase Frontend()
- 2) Indica que el archivo de gui esta en la subcarpeta “DAQ” y se llama “daq_selector.ui”
- 3) Define el método setupUi, que es llamado cuando se inicializa la clase.
- 4) Le pone el texto al checkbox ui, usando el string de virtual daq. (Nota sobre la localización en el apéndice).
- 5) Verifica en el archivo de configuración si se uso previamente el daq virtual:
- 6) Si esto es verdad, lo configura como estado default
- 7) Si esto es falso, configura como default el daq real.

Apéndice:

Sobre la localización del texto:

El texto que se presenta al usuario tiene que ser flexible para poder adaptar multiples idiomas. Para reflejar esto, usamos un sistema de localización. En el archivo “View\localization.py” se encuentran diccionarios para los distintos idiomas soportados por el programa. Ejemplo:

```
locale_en = {
    "box_gain": "Gain",
    "time_constant": "Time Constant",
    "1_ms": "1 ms",
    "high_reserve": "High Reserve",
    "normal": "Normal",
    "low_noise": "Low Noise",
}
```

Para llamar a la localización usamos el método `locale.get("string", "fallback")`, donde "string" es el nombre de lo que queremos buscar y "fallback" es lo que usamos si no se encuentra la localización. El standard que uso es "str_string", para indicar que falta esa localización. El código donde inicia el programa revisa el archivo de configuración y los parámetros que se le mandan al programa para elegir que idioma mostrar.

Documentación de QT:

Para revisar que métodos y acciones se pueden realizar con los widgets presentes en la plataforma QT, la documentación principal de QT se puede encontrar en <https://doc.qt.io/> (en ingles).